Gender Detection

A Project Report

Submitted in partial fulfillment of the

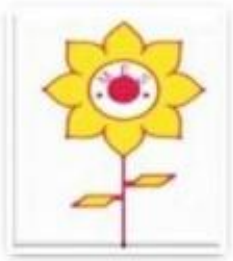
Requirements for the award of the Degree of

MASTER OF SCIENCE (DATA ANALYTICS)

Name of the Student: Jayesh Jadhav
Seat Number: 6916

Under the esteemed guidance of Mr. OMKAR SHERKHANE

Designation: Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE

MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Autonomous), NEW PANVEL, 410206, MAHARASHTRA

(*Affiliated to University of Mumbai)*

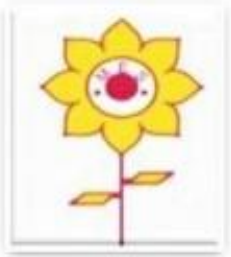
2024-2025

MAHATMA EDUCATION SOCIETY'S PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Autonomous) NEW PANVEL, 410206, MAHARASHTRA

(Affiliated to University of Mumbai)

DEPARTMENT OF COMPUTER SCIENCE



CERTIFICATE

This is to certify that the project entitled **"Gender Detection "** is Bonafide work of **Jayesh Jadhav** bearing Roll. No **6916** submitted in fulfillment for the completion of MSc. degree in Data Analytics of University of Mumbai.

Internal Guide **Co-Ordinator**

Date: **College seal** **External Examiner**

ACKNOWLEDGEMENT

I Mr.Jayesh Jadhav student of PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE (AUTONOMOUS), NEW PANVEL would like to express my sincere gratitude towards our college's Computer Science Department.

I would like to thank our Coordinator Mr. Omkar Sherkhane for his constant support during this project and granting me the opportunity to build a project for the college. The project would have not been completed without the dedication, creativity and the enthusiasm my family provided me.

Yours faithfully,
Jayesh Jadhav

DECLARATION

I hereby, declare that the project entitled, "**Gender Detection**" done at Pillai College of Arts, Commerce & Science (Autonomous), New Panvel, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in fulfillment of the requirements for the award of degree of MASTER OF SCIENCE (DATA ANALYTICS) to be submitted as a final semester project as part of our curriculum.

Signature of the Student

ABSTRACT

This project presents a gender detection system that leverages deep learning and computer vision techniques to classify faces as either male or female. The model is trained using a convolutional neural network (CNN), which processes a dataset of labeled face images. The system is designed with a user-friendly interface, developed with Flask, allowing users to interact with the model by either uploading an image for gender prediction or using a live camera feed for real-time classification.

The dataset comprises face images categorized into two classes: "man" and "woman." Image preprocessing, including resizing and normalization, ensures the model handles diverse input formats. The deep learning architecture consists of multiple convolutional layers with ReLU activation, batch normalization, and max-pooling layers to extract and refine features. Dropout layers are used to mitigate overfitting. The model is compiled with binary cross-entropy as the loss function and the Adam optimizer to enhance performance over 100 epochs.

The Flask-based web application provides two primary functionalities: image upload and real-time prediction using a webcam. For the upload feature, the system accepts an image, processes it, and predicts the gender along with confidence levels. The live camera feed feature allows users to see gender predictions overlaid on faces in real-time. The results are displayed on a web page, making the system accessible and intuitive.

The project's potential applications include demographic analysis, human-computer interaction, and intelligent marketing systems. The integration of TensorFlow for model training, OpenCV for face detection, and Flask for deployment demonstrates the seamless fusion of machine learning with real-time applications. Future enhancements could involve expanding the model to recognize non-binary gender identities or improving accuracy with more diverse datasets.

Index

Chapter 1

Introduction

In today's era of artificial intelligence (AI), gender detection from facial images is becoming an increasingly relevant application, used in areas such as personalized marketing, biometric systems, and social media analytics. This project focuses on developing a gender detection system using deep learning techniques, specifically a Convolutional Neural Network (CNN), to classify images as either male or female.

The project leverages a CNN model, known for its effectiveness in image classification tasks, to learn and extract features from facial images. By training the model on a labeled dataset containing images of men and women, the system can predict gender based on new, unseen images. The model architecture includes multiple convolutional layers for feature extraction, along with max-pooling and dropout layers to improve accuracy and prevent overfitting.

The system provides two modes of interaction: users can upload an image or use a live webcam feed to predict gender in real-time. Flask, a lightweight web framework, is used for the backend, while OpenCV handles image preprocessing, including face detection and resizing. The combination of these technologies ensures an efficient and user-friendly interface.

The application not only demonstrates the power of deep learning in solving real-world problems but also highlights the importance of accurate and reliable facial recognition systems in modern technology. With further development, the project can be extended to include non-binary gender classifications and additional functionalities for broader applications.

Chapter 2

Review of Literature

2.1 Background

Gender detection using AI and machine learning is a fast-growing field that applies image processing and classification techniques to determine an individual's gender based on visual data. In numerous industries, gender recognition plays a critical role, such as targeted marketing, surveillance, security systems, and personalized content delivery. Leveraging deep learning models, particularly Convolutional Neural Networks (CNNs), offers a high level of accuracy in image-based tasks, making it a popular choice for gender classification tasks.

2.2 Objectives

The primary objective of this project is to develop a robust and accurate gender detection system that can classify facial images as either male or female. Specific goals include:

1. Designing and training a CNN model capable of recognizing gender features from facial images.

2. Developing a user-friendly graphical interface that allows users to upload images or use a live webcam feed for real-time predictions.

3. Achieving a high accuracy rate by optimizing the model through image augmentation, batch normalization, and dropout techniques.

4. Integrating the model into a web-based platform using Flask for easy accessibility and deployment.

2.3 Purpose

The purpose of this project is to explore and implement modern AI techniques for practical applications in gender detection. By building a gender classification system, this project demonstrates how deep learning can be applied to solve real-world problems. The project aims to familiarize developers with CNN architectures, face detection algorithms, and image preprocessing techniques, while offering an interactive experience through a web-based platform.

2.4 Scope of the project

This project focuses on:

1. Building a CNN model for binary gender classification (male and female).
2. Integrating the model into a web application that provides two functionalities: image upload and real-time webcam-based predictions.
3. Implementing basic face detection and preprocessing using OpenCV to improve prediction accuracy.
4. Training and testing the model on a gender-labeled dataset of facial images.

2.5 Applicability

The gender detection system developed in this project has a wide range of potential applications:

1. **Marketing and Advertising**: Companies can use gender information to target content or advertisements to specific demographics, enhancing personalized experiences.
2. **Security and Surveillance**: Gender recognition systems can assist in monitoring and identifying individuals in public spaces for enhanced security.
3. **Human-Computer Interaction**: The system can be applied to optimize user interfaces and customize interactions based on the user's gender.
4. **Biometric Authentication**: It can serve as a component in broader biometric systems used for access control or identity verification.
5. **Social Media and Content Filtering**: Platforms can implement gender detection to analyze user data and offer more tailored recommendations.

Chapter 3

System Planning

Survey of Technologies

3.1 Front End

- **HTML**:
    - HTML (HyperText Markup Language) is used to structure the web pages in the project. It defines the layout for the gender detection interface, including forms for image upload, live camera streaming, and result display.
- **CSS**:
    - CSS (Cascading Style Sheets) is used to style the HTML pages. It ensures the web pages are visually appealing, responsive, and provide a smooth user experience. CSS is used for animations, button styles, form layouts, and page aesthetics.
- **JavaScript** (for interactivity, if applicable):
    - Though not explicitly included in the code, JavaScript can be used for front-end interactions like form validation or dynamic updates without reloading the page (if necessary).
- **Jinja2 Templates**:
    - Flask uses Jinja2 templating to dynamically render content on HTML pages. The uploaded image path and prediction results are inserted into the HTML templates using Jinja2, ensuring seamless communication between the front end and back end.

Languages Used:

- **Python**:
    - Python is the primary programming language used in this project. It is used for building the machine learning model (using TensorFlow/Keras), developing the Flask backend, handling image processing with OpenCV, and integrating everything together.
- **HTML**:
    - HTML is used for structuring the web pages and forms for the web interface.
- **CSS**:
    - CSS is used for styling the front-end user interface.

3.2 Back End

- **Flask**:
    - Flask is the core web framework used for backend development. It handles routing, form submission, processing uploaded images, and rendering dynamic web pages. Flask communicates with the machine learning model and serves the prediction results to the front-end interface.
- **TensorFlow/Keras**:
    - These libraries are used in the backend to load the pre-trained CNN model and perform predictions on the uploaded images or real-time webcam frames. The backend is responsible for calling the model, processing inputs, and returning gender predictions.
- **OpenCV (cv2)**:
    - OpenCV is used in the backend for image and video processing, including reading, resizing, and preprocessing images before passing them to the model for prediction. It also handles real-time video frame processing for live gender detection.
- **cvlib**:
    - cvlib simplifies face detection within the live camera feed. It works in the backend to detect faces in frames and sends the detected regions for gender classification.

3.3 Fact Finding Techniques

1. Observation

- **Description**: Involves directly watching and noting how users interact with existing systems or processes. This technique helps in understanding real-world behaviors, workflows, and challenges.
- **Purpose**: To gather firsthand knowledge about how tasks are currently performed, which may reveal inefficiencies or areas for improvement that the users themselves may not report.
- **Example in Project**: Observing how people upload and interact with the gender detection model, or how they use live camera feeds for real-time predictions.

2. Reviewing Existing Documents

- **Description**: Examining current documentation, including system manuals, reports, business process documents, and software documentation, to understand the existing environment.
- **Purpose**: To gain insights into how the system or processes are structured, identify key pain points, and understand the history of the existing setup.
- **Example in Project**: Reviewing existing literature on gender detection, similar applications, or reports about the challenges of real-time image processing.

3. Feasibility Study

- **Description**: An analysis of the viability of the project in terms of technical, operational, and economic feasibility. This study helps determine whether the proposed solution can be realistically implemented.
- **Types of Feasibility**:
    - **Technical Feasibility**: Whether the technology required for gender detection (CNN, TensorFlow, Flask) is available and can be implemented.
    - **Operational Feasibility**: Whether the project meets the operational needs of stakeholders and users.
    - **Economic Feasibility**: Whether the costs of implementation are justified by the benefits.
- **Purpose**: To determine if the project is practical, cost-effective, and worth pursuing.
- **Example in Project**: Evaluating if the existing resources, including hardware for real-time video processing and model inference, are sufficient for smooth functioning.

4. Stakeholders Analysis

- **Description**: Identifying all individuals, groups, or organizations that have an interest in the project. Stakeholders can include users, system administrators, developers, project managers, and external parties.
- **Purpose**: To ensure all relevant viewpoints are considered, gather requirements, and determine how the project's outcome will affect them.
- **Example in Project**: Identifying users who will benefit from the gender detection system, including both technical users (developers) and non-technical users (end-users who interact with the application).

Chapter 4 Approach & Methodology

4.1 Approach

Convolutional Neural Network (CNN) approach was used, combined with a machine learning pipeline that includes data preprocessing, training, validation, and deployment of the model in a web-based application. The CNN model was chosen because of its proven effectiveness in image recognition tasks, making it ideal for detecting gender based on facial images.

Steps in the Process

1. **Data Collection and Preprocessing**
    - **Step**: A large dataset of facial images labeled by gender (male/female) was collected. The images were then resized and normalized to prepare them for input into the CNN.
    - **Application**: The data preparation step ensures that the input images are uniform in size and scale, enhancing the model's ability to detect relevant features.
2. **Model Design (CNN Architecture)**
    - **Step**: A CNN architecture was designed to extract relevant features from the input images. Key layers such as convolutional layers, pooling layers, and fully connected layers were used to detect gender from facial images.
    - **Application**: CNN automatically extracts features like edges, textures, and shapes from images, which are essential for gender classification.
3. **Training the Model**
    - **Step**: The CNN model was trained using labeled data. The model learns patterns associated with male and female facial features. Backpropagation and gradient descent were used to minimize the error during training.
    - **Application**: By training on thousands of images, the CNN model becomes capable of recognizing distinguishing features between genders.
4. **Validation and Testing**
    - **Step**: The trained model was validated on a separate dataset to evaluate its accuracy, precision, recall, and F1-score. Hyperparameters were adjusted during this phase to improve performance.
    - **Application**: Validation ensures that the model generalizes well to unseen data, preventing overfitting and ensuring robust predictions.
5. **Model Deployment**
    - **Step**: The trained CNN model was deployed using Flask (backend) and integrated with a user-friendly GUI built using Tkinter. The GUI allows users to upload an image or use a live camera feed to predict gender in real-time.
    - **Application**: The deployment makes the model accessible to end-users, who can interact with the gender detection system through a simple interface.
6. **Application Integration and Testing**
    - **Step**: The system was tested with real-world input data to evaluate its speed and accuracy in predicting gender. The application's performance was monitored, and adjustments were made for optimization.
    - **Application**: Real-time testing helps refine the system for practical use and ensures a seamless user experience.

Applications of the Approach

1. **Real-Time Gender Detection**: The system allows users to upload facial images or use a live camera feed for real-time gender prediction.
2. **Security Systems**: This approach can be integrated into facial recognition systems used in security applications, enabling gender identification for profiling purposes.

3. **Marketing and User Personalization**: Gender detection can be used to provide tailored content or personalized recommendations based on the detected gender of users, such as in retail or online advertising.
4. **Surveillance Systems**: In public safety applications, the gender detection feature can be incorporated into surveillance systems to monitor and analyze crowd demographics.
5. **Research and Analytics**: The approach can be applied in research studies to analyze gender-based behavioral patterns or demographic distributions in various fields.

Chapter 5 Requirement & Analysis

5.1 Problem Definition

In the contemporary digital age, gender detection plays a crucial role in various applications such as security, marketing, and user analytics. Traditional methods of gender detection rely heavily on manual input, which can lead to inaccuracies and inefficiencies. The objective of this project is to develop an automated gender detection system that accurately classifies individuals based on their facial images using machine learning techniques, specifically Convolutional Neural Networks (CNNs). This system aims to provide real-time predictions while ensuring high accuracy and usability.

5.2 Requirement Specification

Functional Requirements

1. **Image Input**: The system should allow users to upload facial images for gender detection or utilize a live camera feed.
2. **Gender Classification**: The system must predict the gender of the person in the image (male or female).
3. **Display Results**: The system should display the predicted gender and confidence level of the prediction to the user.
4. **User Interface**: A simple and intuitive GUI should be provided for seamless user interaction.

Non-Functional Requirements

1. **Accuracy**: The model should achieve a prediction accuracy of at least 85%.
2. **Speed**: The system should provide predictions in real-time or within a few seconds.
3. **Scalability**: The system should handle multiple users simultaneously without performance degradation.
4. **Usability**: The GUI should be user-friendly and accessible to individuals with minimal technical knowledge.

5.3 Modules of Proposed System

- **Data Acquisition Module**
  - Responsible for collecting and preprocessing images.
  - Includes functions for image uploading and real-time camera access.
- **Model Training Module**
  - Contains the implementation of the CNN architecture.
  - Handles the training process, including data augmentation, hyperparameter tuning, and model validation.
- **Prediction Module**
  - Utilizes the trained model to make predictions on new images.
  - Processes input images and returns predicted gender and confidence levels.

- **User Interface Module**
    - Provides the front-end GUI developed with Tkinter.
    - Displays results and allows user interactions, including image uploads and camera access.
- **Reporting Module**
    - Generates reports on prediction accuracy and user interactions.
    - Allows users to review previous predictions if required.

Chapter 6

6.1 Program Code

- **model_train.py**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import img_to_array
from tensorflow.keras.utils import to_categorical, plot_model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import BatchNormalization, Conv2D, MaxPooling2D, Activation, Flatten, Dropout,
Dense
from tensorflow.keras import backend as K
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np
import random
import cv2
import os
import glob

# initial parameters
epochs = 100
lr = 1e-3
batch_size = 64
img_dims = (96,96,3)

data = []
labels = []

# load image files from the dataset
image_files = [f for f in glob.glob(r'C:\Files\gender_dataset_face' + "/**/*", recursive=True) if not os.path.isdir(f)]
random.shuffle(image_files)

# converting images to arrays and labelling the categories
for img in image_files:

    image = cv2.imread(img)

    image = cv2.resize(image, (img_dims[0],img_dims[1]))
    image = img_to_array(image)
    data.append(image)

    label = img.split(os.path.sep)[-2] # C:\Files\gender_dataset_face\woman\face_1162.jpg
    if label == "woman":
        label = 1
    else:
        label = 0

    labels.append([label]) # [[1], [0], [0], ...]

# pre-processing
data = np.array(data, dtype="float") / 255.0
labels = np.array(labels)

# split dataset for training and validation
(trainX, testX, trainY, testY) = train_test_split(data, labels, test_size=0.2,
                                  random_state=42)
```

```
trainY = to_categorical(trainY, num_classes=2) # [[1, 0], [0, 1], [0, 1], ...]
testY = to_categorical(testY, num_classes=2)

# augmenting datset
aug = ImageDataGenerator(rotation_range=25, width_shift_range=0.1,
                         height_shift_range=0.1, shear_range=0.2, zoom_range=0.2,
                         horizontal_flip=True, fill_mode="nearest")

# define model
def build(width, height, depth, classes):
    model = Sequential()
    inputShape = (height, width, depth)
    chanDim = -1

    if K.image_data_format() == "channels_first": #Returns a string, either 'channels_first' or 'channels_last'
        inputShape = (depth, height, width)
        chanDim = 1

    # The axis that should be normalized, after a Conv2D layer with data_format="channels_first",
    # set axis=1 in BatchNormalization.

    model.add(Conv2D(32, (3,3), padding="same", input_shape=inputShape))
    model.add(Activation("relu"))
    model.add(BatchNormalization(axis=chanDim))
    model.add(MaxPooling2D(pool_size=(3,3)))
    model.add(Dropout(0.25))

    model.add(Conv2D(64, (3,3), padding="same"))
    model.add(Activation("relu"))
    model.add(BatchNormalization(axis=chanDim))

    model.add(Conv2D(64, (3,3), padding="same"))
    model.add(Activation("relu"))
    model.add(BatchNormalization(axis=chanDim))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Dropout(0.25))

    model.add(Conv2D(128, (3,3), padding="same"))
    model.add(Activation("relu"))
    model.add(BatchNormalization(axis=chanDim))

    model.add(Conv2D(128, (3,3), padding="same"))
    model.add(Activation("relu"))
    model.add(BatchNormalization(axis=chanDim))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Dropout(0.25))

    model.add(Flatten())
    model.add(Dense(1024))
    model.add(Activation("relu"))
    model.add(BatchNormalization())
    model.add(Dropout(0.5))

    model.add(Dense(classes))
    model.add(Activation("sigmoid"))

    return model

# build model
model = build(width=img_dims[0], height=img_dims[1], depth=img_dims[2],
```

```
                    classes=2)

# compile the model
opt = Adam(lr=lr, decay=lr/epochs)
model.compile(loss="binary_crossentropy", optimizer=opt, metrics=["accuracy"])

# train the model
H = model.fit_generator(aug.flow(trainX, trainY, batch_size=batch_size),
                    validation_data=(testX,testY),
                    steps_per_epoch=len(trainX) // batch_size,
                    epochs=epochs, verbose=1)

# save the model to disk
model.save('gender_detection_model.keras')

# plot training/validation loss/accuracy
plt.style.use("ggplot")
plt.figure()
N = epochs
plt.plot(np.arange(0,N), H.history["loss"], label="train_loss")
plt.plot(np.arange(0,N), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0,N), H.history["acc"], label="train_acc")
plt.plot(np.arange(0,N), H.history["val_acc"], label="val_acc")

plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend(loc="upper right")

# save plot to disk
plt.savefig('plot.png')
```

- **gui.py**

```
from flask import Flask, render_template, request, redirect, url_for, Response
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import img_to_array
import numpy as np
import cv2
import cvlib as cv
import os

# Initialize Flask app
app = Flask(__name__)

# Load the pre-trained model
model = load_model('gender_detection_model.keras')
classes = ['man', 'woman']

# Route for homepage
@app.route('/')
def index():
    return render_template('index.html')  # HTML page with upload and camera options

# Route for handling image upload and prediction
@app.route('/upload', methods=['POST'])
def upload():
    if 'file' not in request.files:
```

```
        return redirect(request.url)
    file = request.files['file']

    if file.filename == '':
        return redirect(request.url)

    if file:
        # Create the directory if it doesn't exist
        upload_folder = "static/uploads"
        if not os.path.exists(upload_folder):
            os.makedirs(upload_folder)  # Create the directory if it doesn't exist

        # Save the uploaded image
        filepath = os.path.join(upload_folder, file.filename)
        file.save(filepath)

        # Load and preprocess the image for prediction
        img = cv2.imread(filepath)
        img = cv2.resize(img, (96, 96))
        img = img.astype("float") / 255.0
        img = img_to_array(img)
        img = np.expand_dims(img, axis=0)

        # Perform prediction
        conf = model.predict(img)[0]
        idx = np.argmax(conf)
        label = classes[idx]
        confidence = conf[idx] * 100

        # Render result
        return render_template('result.html', label=label, confidence=confidence, filepath=filepath)


# Route for handling live camera feed
@app.route('/live_camera')
def live_camera():
    return render_template('camera.html')

def gen_frames():
    # Open the webcam
    webcam = cv2.VideoCapture(0)

    while True:
        success, frame = webcam.read()
        if not success:
            break
        else:
            face, confidence = cv.detect_face(frame)

            # Loop through detected faces
            for idx, f in enumerate(face):
                (startX, startY) = f[0], f[1]
                (endX, endY) = f[2], f[3]

                # Draw rectangle over face
                cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)

                # Crop face and preprocess it
                face_crop = np.copy(frame[startY:endY, startX:endX])
                if (face_crop.shape[0]) < 10 or (face_crop.shape[1]) < 10:
```

```
                continue
            face_crop = cv2.resize(face_crop, (96, 96))
            face_crop = face_crop.astype("float") / 255.0
            face_crop = img_to_array(face_crop)
            face_crop = np.expand_dims(face_crop, axis=0)

            # Perform gender prediction
            conf = model.predict(face_crop)[0]
            idx = np.argmax(conf)
            label = classes[idx]
            label = "{}: {:.2f}%".format(label, conf[idx] * 100)

            # Display label
            Y = startY - 10 if startY - 10 > 10 else startY + 10
            cv2.putText(frame, label, (startX, Y), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        # Encode the frame in JPEG format
        ret, buffer = cv2.imencode('.jpg', frame)
        frame = buffer.tobytes()

        # Yield the frame in byte format for streaming
        yield (b'--frame\r\n' b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

    webcam.release()

@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

if __name__ == '__main__':
    app.run(debug=True)
```

- **templates\index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gender Detection</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
      padding: 0;
    }

    h1 {
      font-size: 3rem;
      color: #333;
      margin-bottom: 1rem;
      animation: fadeIn 2s ease-in-out;
    }

    h3 {
```

```
    font-size: 1.5rem;
    color: #666;
    margin-bottom: 1rem;
}

form {
    margin-bottom: 2rem;
    text-align: center;
}

input[type="file"] {
    display: inline-block;
    margin-right: 1rem;
    padding: 0.5rem;
    border: 2px solid #ddd;
    border-radius: 5px;
    background-color: #fff;
    cursor: pointer;
}

button {
    background-color: #4CAF50;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 1rem;
    transition: background-color 0.3s ease, transform 0.2s ease;
}

button:hover {
    background-color: #45a049;
    transform: scale(1.1);
}

a {
    display: inline-block;
    background-color: #2196F3;
    color: white;
    padding: 10px 20px;
    text-decoration: none;
    border-radius: 5px;
    transition: background-color 0.3s ease, transform 0.2s ease;
}

a:hover {
    background-color: #0b7dda;
    transform: scale(1.1);
}

@keyframes fadeIn {
    0% {
        opacity: 0;
        transform: translateY(-20px);
    }
    100% {
        opacity: 1;
        transform: translateY(0);
    }
```

```
        }

        /* Add some space around the page */
        body > * {
            animation: fadeIn 1.5s ease-in-out;
        }
    </style>
</head>
<body>
    <h1>Gender Detection</h1>
    <h3>Upload Image for Prediction</h3>
    <form action="/upload" method="POST" enctype="multipart/form-data">
        <input type="file" name="file">
        <button type="submit">Upload and Predict</button>
    </form>

    <h3>Live Camera Feed</h3>
    <a href="/live_camera">Open Live Camera</a>
</body>
</html>
```

- **camera.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Live Camera Feed</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f0f0f0;
            display: flex;
            flex-direction: column;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            padding: 0;
        }

        h1 {
            font-size: 3rem;
            color: #333;
            margin-bottom: 2rem;
            animation: fadeIn 2s ease-in-out;
        }

        img {
            border: 5px solid #ddd;
            border-radius: 10px;
            box-shadow: 0px 0px 15px rgba(0, 0, 0, 0.2);
            transition: transform 0.3s ease;
        }

        img:hover {
            transform: scale(1.05);
        }
```

```
        a {
            display: inline-block;
            background-color: #2196F3;
            color: white;
            padding: 10px 20px;
            text-decoration: none;
            border-radius: 5px;
            margin-top: 20px;
            transition: background-color 0.3s ease, transform 0.2s ease;
        }

        a:hover {
            background-color: #0b7dda;
            transform: scale(1.1);
        }

        @keyframes fadeIn {
            0% {
                opacity: 0;
                transform: translateY(-20px);
            }
            100% {
                opacity: 1;
                transform: translateY(0);
            }
        }

        /* Smooth animation for content */
        body > * {
            animation: fadeIn 1.5s ease-in-out;
        }
    </style>
</head>
<body>
    <h1>Live Camera Feed</h1>
    <img src="{{ url_for('video_feed') }}" width="640" height="480" alt="Live Camera Feed">
    <br><br>
    <a href="/">Go Back</a>
</body>
</html>
```

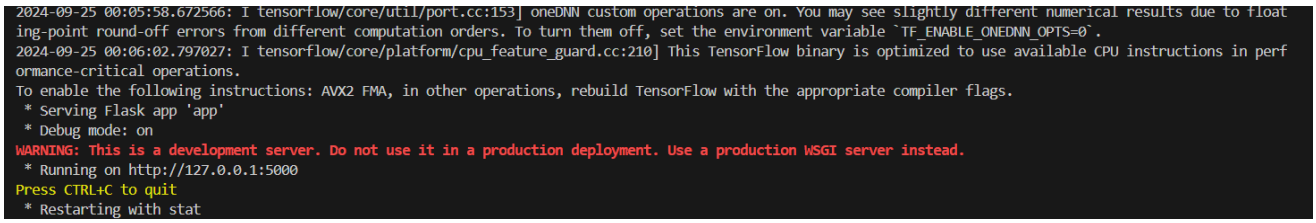
- **result.html**

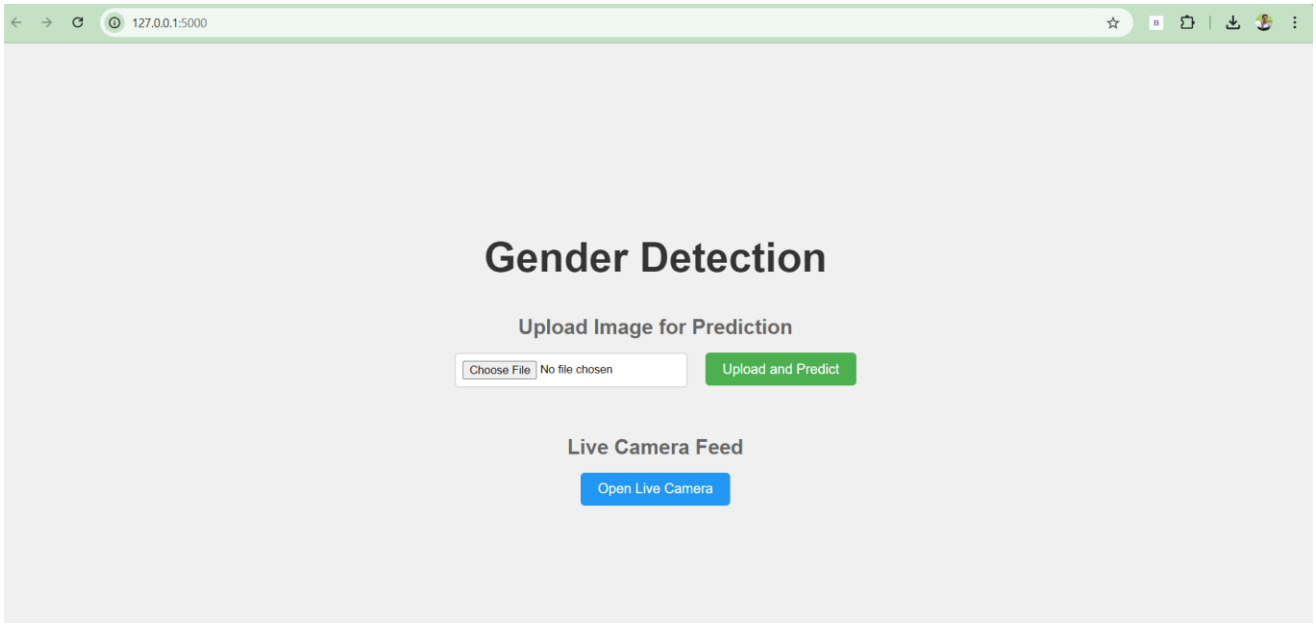
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Prediction Result</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f9f9f9;
            display: flex;
            flex-direction: column;
            justify-content: center;
            align-items: center;
```

```
    height: 100vh;
    margin: 0;
    padding: 0;
}

h1 {
    font-size: 3rem;
    color: #333;
    margin-bottom: 1.5rem;
    animation: fadeIn 1.5s ease-in-out;
}

p {
    font-size: 1.5rem;
    color: #555;
    margin-bottom: 1rem;
    animation: fadeIn 2s ease-in-out;
}

img {
    max-width: 400px;
    border: 4px solid #ddd;
    border-radius: 10px;
    box-shadow: 0px 0px 15px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease;
    animation: fadeIn 2.5s ease-in-out;
}

img:hover {
    transform: scale(1.05);
}

a {
    display: inline-block;
    background-color: #4CAF50;
    color: white;
    padding: 10px 20px;
    text-decoration: none;
    border-radius: 5px;
    margin-top: 20px;
    transition: background-color 0.3s ease, transform 0.2s ease;
}

a:hover {
    background-color: #45a049;
    transform: scale(1.1);
}

@keyframes fadeIn {
    0% {
        opacity: 0;
        transform: translateY(-20px);
    }
    100% {
        opacity: 1;
        transform: translateY(0);
    }
}

/* Smooth fade-in for all content */
```

```
        body > * {
            animation: fadeIn 1.5s ease-in-out;
        }
    </style>
</head>
<body>
    <h1>Prediction Result</h1>
    <p>Predicted Gender: <strong>{{ label }}</strong></p>
    <p>Confidence: <strong>{{ confidence }}%</strong></p>
    <img src="{{ url_for('static', filename=filepath) }}" alt="Uploaded Image">
    <br><br>
    <a href="/">Go Back</a>
</body>
</html>
```

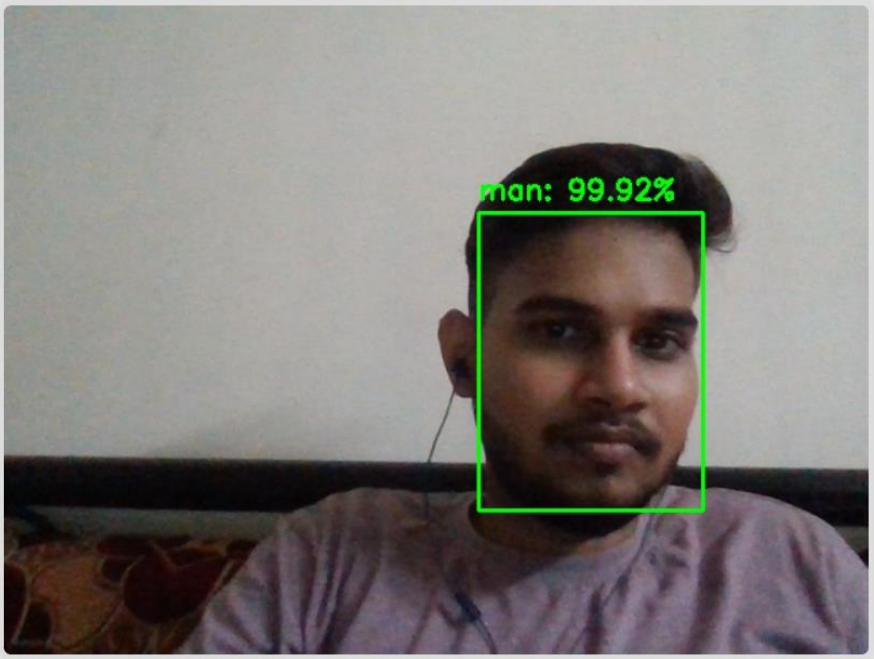
6.2 Output Images





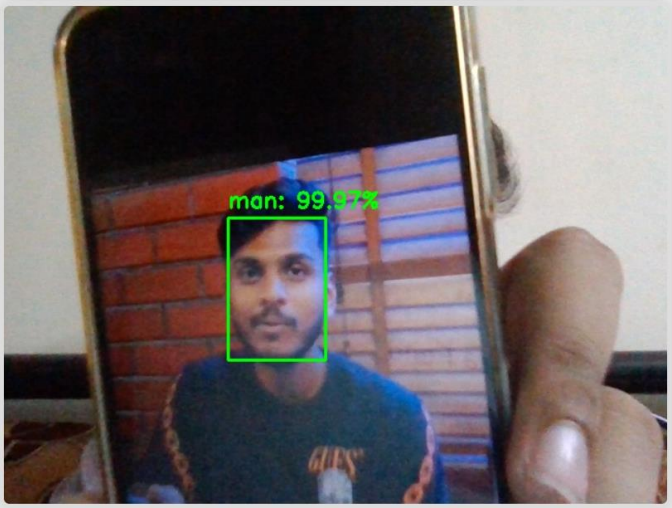
Live Camera Result:-

Live Camera Feed

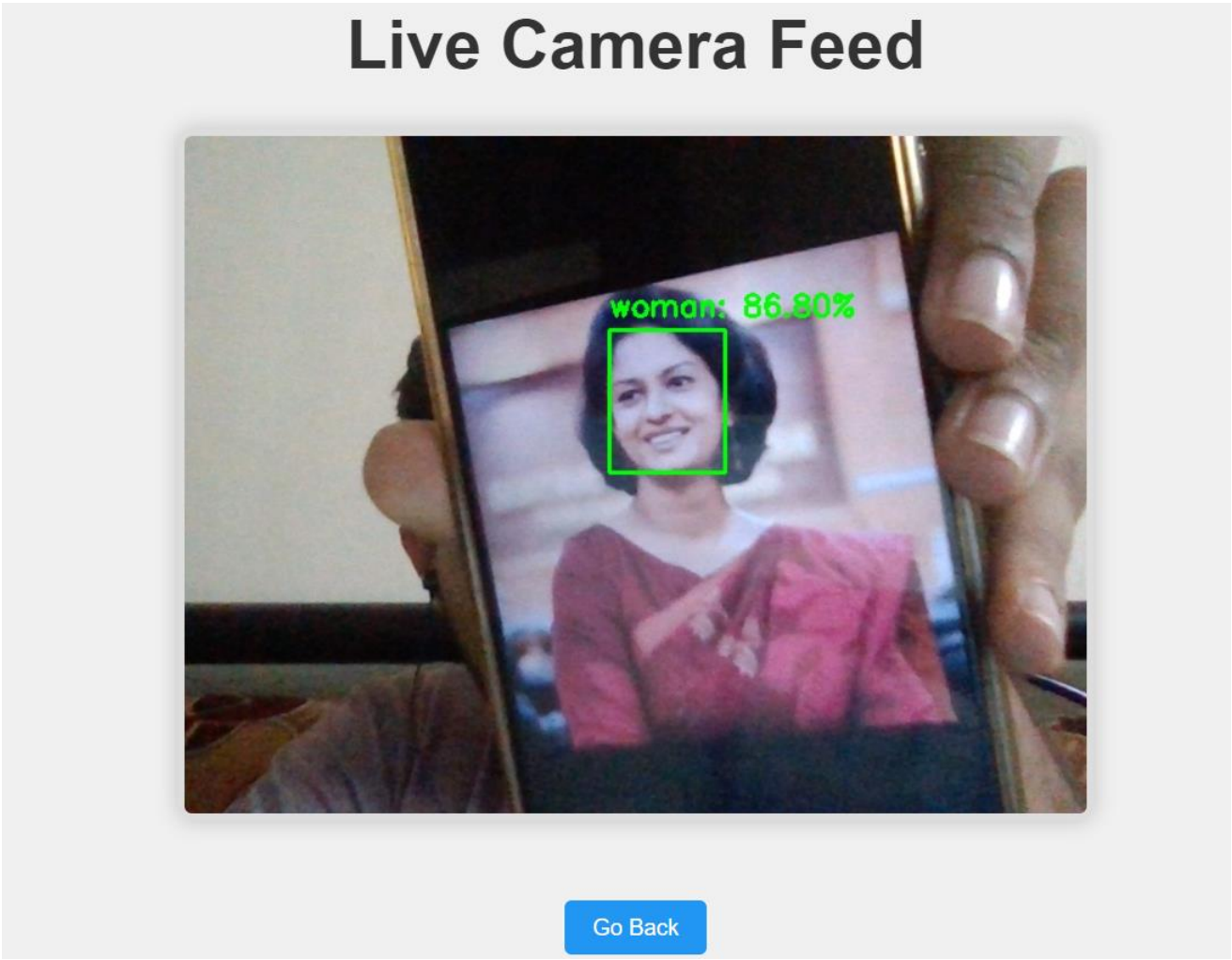


Go Back

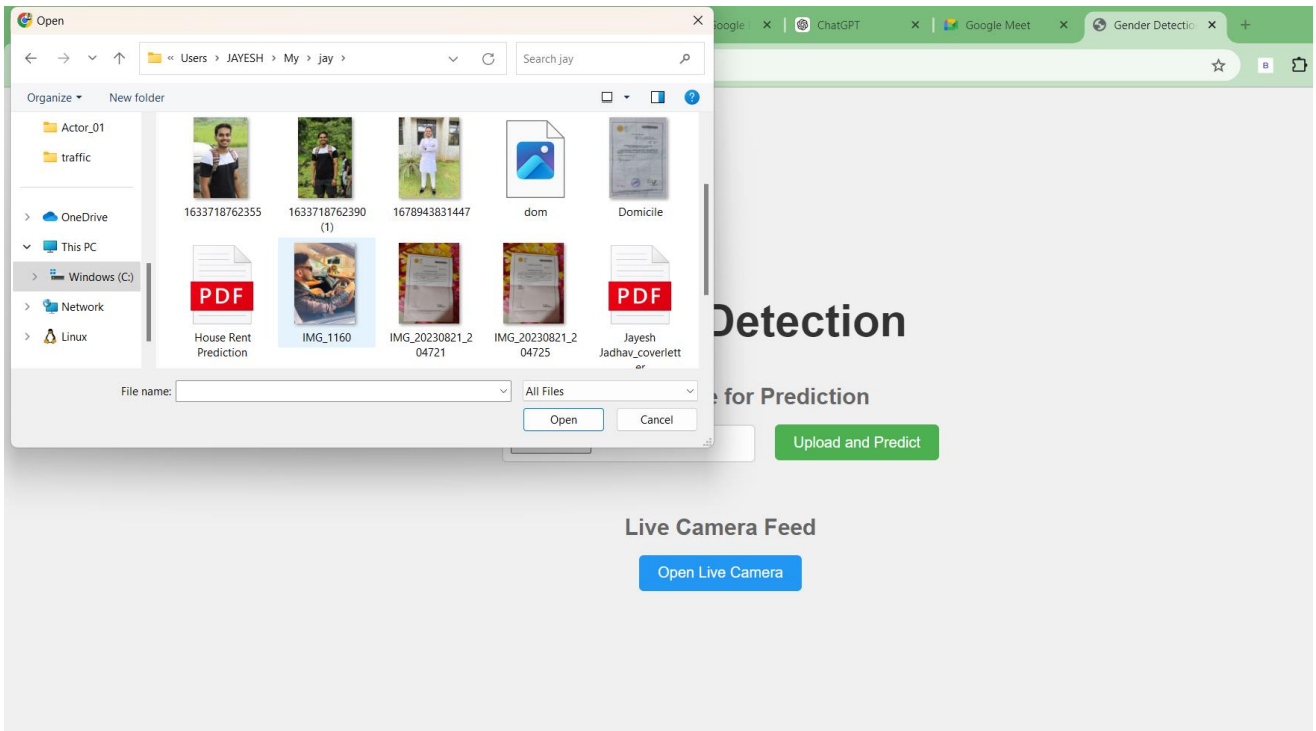
Live Camera Feed

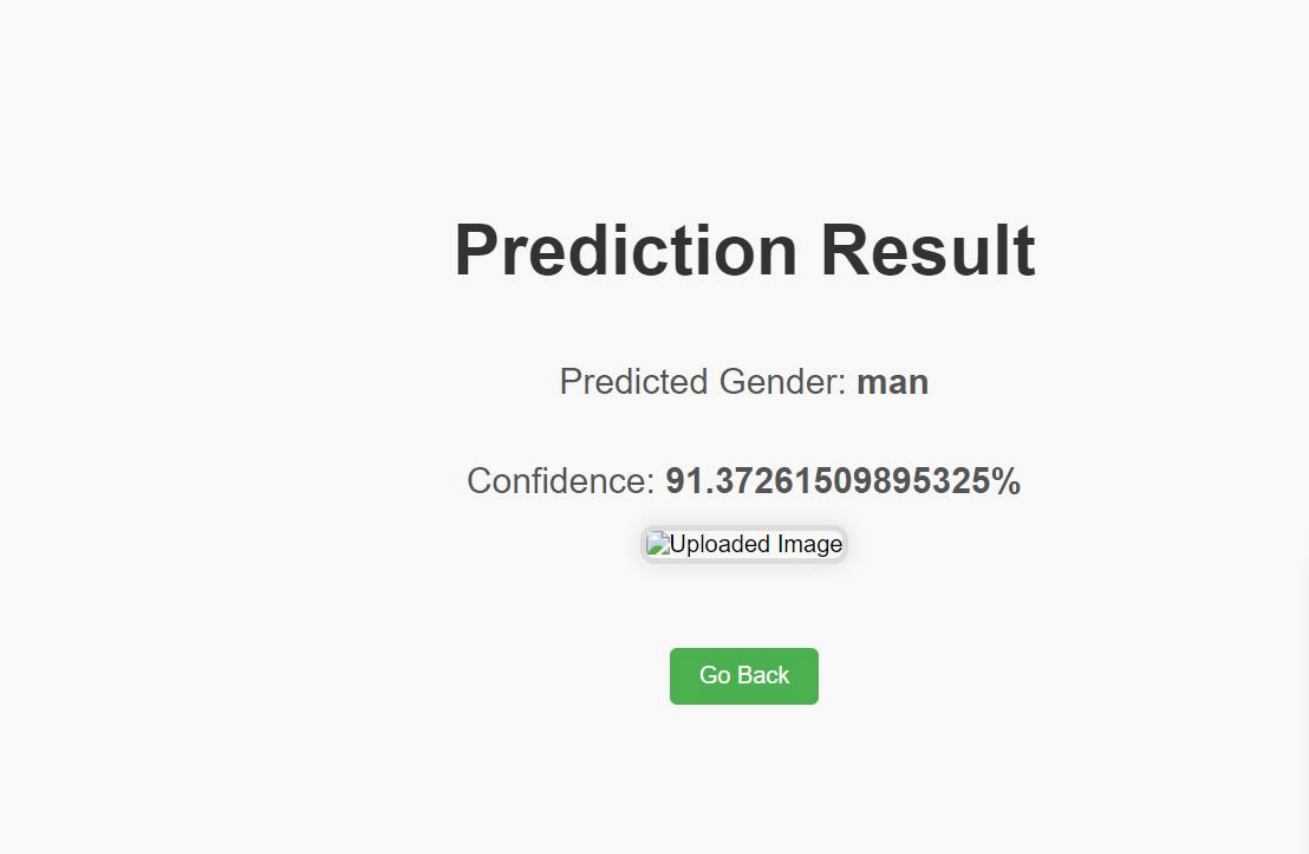


Go Back



Upload image result:-





6.3 Testing Approach

- **Unit Testing**
    - Each module of the system (Data Acquisition, Model Training, Prediction, User Interface, Reporting) will be tested individually to ensure that it functions correctly.
    - Automated tests will be written for functions related to image processing and gender classification.
- **Integration Testing**
    - After unit testing, the interaction between modules will be tested to verify that they work together seamlessly.
    - For instance, the flow of data from the User Interface Module to the Prediction Module will be tested to ensure the output is displayed correctly.
- **System Testing**
    - The entire system will be tested in a controlled environment to assess its performance under various scenarios.
    - Functional testing will ensure that all requirements are met, while non-functional testing will evaluate the system's performance, scalability, and security.
- **User Acceptance Testing (UAT)**

- End-users will be involved in testing the system to ensure it meets their expectations and is user-friendly.
- Feedback will be collected to identify any usability issues or areas for improvement.

- **Performance Testing**

  - The system’s speed and accuracy will be tested under different loads to ensure it can handle multiple users and large datasets without degradation.
  - Stress testing will be performed to evaluate the system’s behavior under extreme conditions.

Chapter 7 Conclusion & Future Scope

7.1 Conclusion

The gender detection project successfully demonstrates the application of machine learning, specifically Convolutional Neural Networks (CNNs), to classify gender from facial images. The system integrates a user-friendly GUI that allows seamless interaction, making it accessible to non-technical users. Throughout the development process, careful consideration was given to functional and non-functional requirements, ensuring that the system is both accurate and efficient. The rigorous testing approach undertaken validates the system's reliability and performance, highlighting its potential for real-world applications in areas such as marketing, security, and user analytics. This project not only contributes to the field of computer vision and machine learning but also sets a foundation for further enhancements and developments.

7.2 Future Scope

- **Real-time Video Processing**
    - Implementing real-time gender detection from video feeds, allowing applications in live environments such as surveillance and marketing analytics.
- **Improved Accuracy and Robustness**
    - Further refining the CNN architecture and exploring advanced techniques like transfer learning to improve accuracy and robustness against diverse input images.
- **Integration with Other Technologies**
    - Integrating the gender detection system with other applications, such as chatbots or customer relationship management (CRM) systems, to provide personalized user experiences based on gender classification.
- **Privacy and Ethical Considerations**
    - Addressing privacy concerns and ethical implications related to gender detection technologies, ensuring compliance with regulations and promoting responsible AI usage.
- **Cross-cultural Adaptability**
    - Conducting studies to adapt the model for different ethnicities and demographics to enhance its applicability in diverse populations.